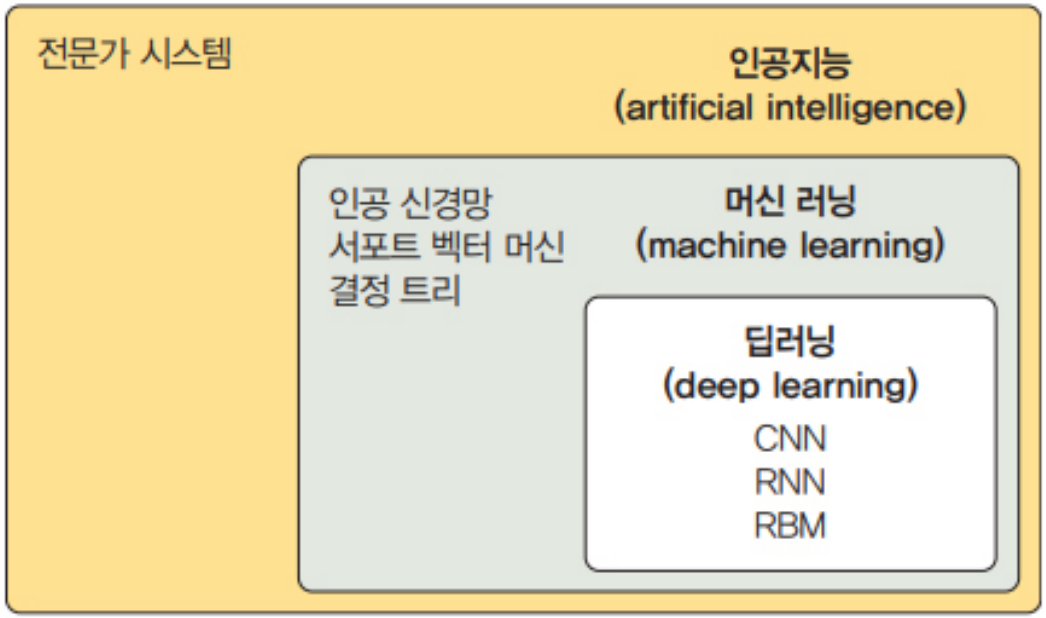
기계학습 프로그래밍

Pytorch #1

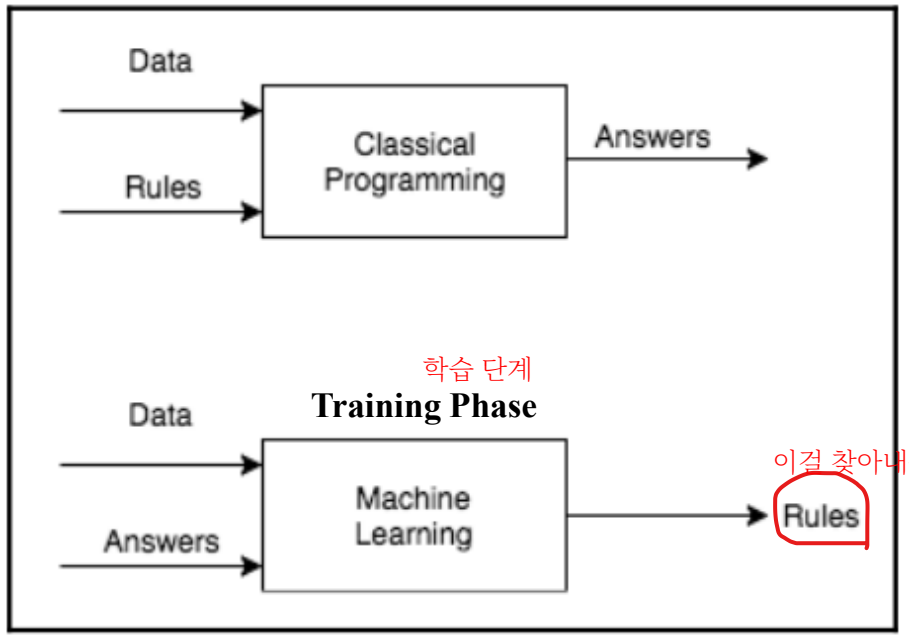
© KOPO-Dept.
AI Engineering Limited

DL. Vs. ML. vs. AI

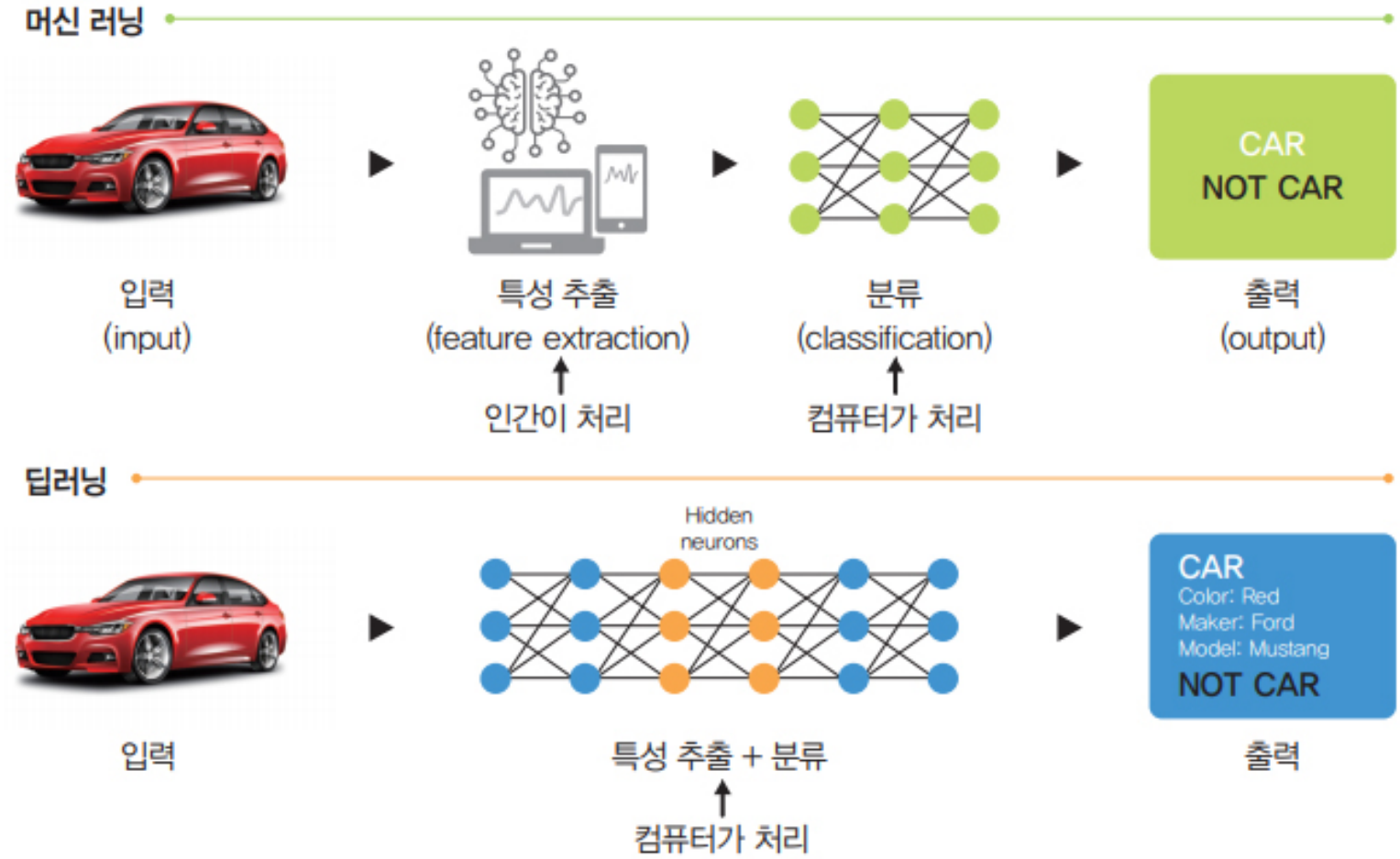
- Deep Learning < Machine Learning < AI(Artificial Intelligence)



Machine Learning



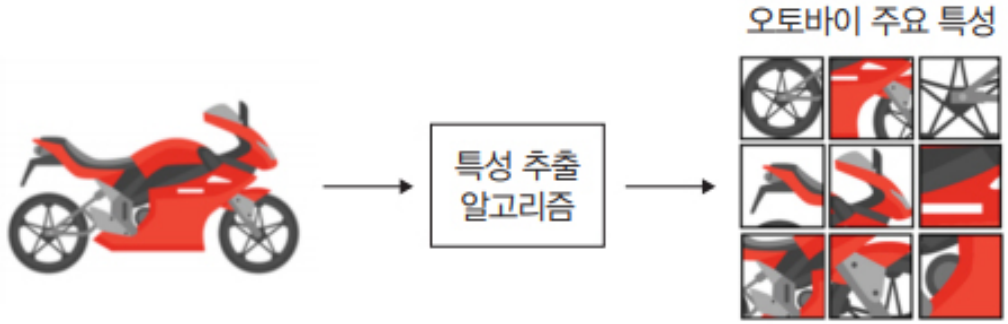
ML vs. Deep Learning



ML vs. Deep Learning

- Feature Extraction (특징(특성) 추출)
  - 머신 러닝에서 컴퓨터가 스스로 학습하려면, 즉 컴퓨터가 **입력받은 데이터를 분석하여 일정한 패턴이나 규칙을 찾아 내려면 사람이 인지하는 데이터를 컴퓨터가 인지할 수 있는 데이터로 변환**해 주어야 함

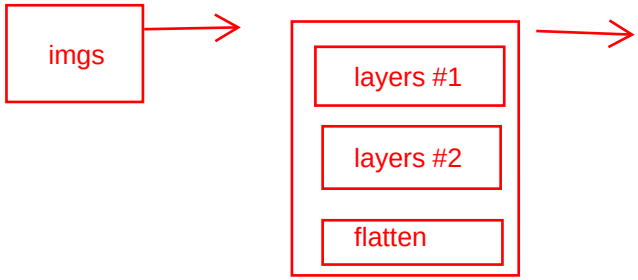
  - 이때 데이터별로 어떤 특징을 가지고 있는지 찾아내고, 그것을 토대로 데이터를 벡터로 변환하는 작업을 특성 추출(feature extraction)이라고 함

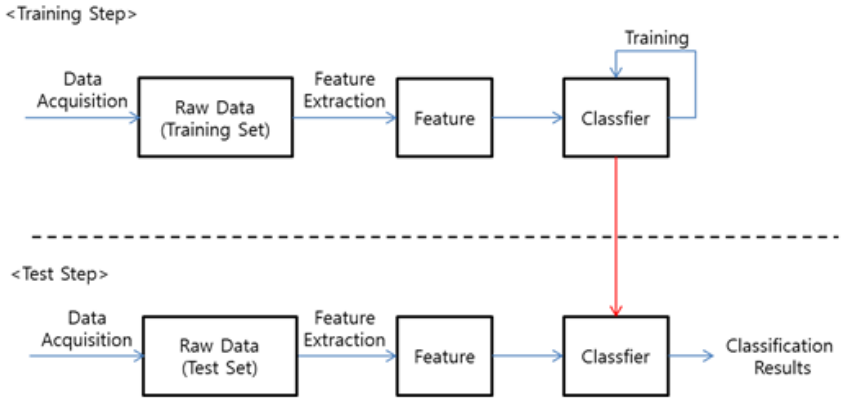


Daegu Campus of KOREA POLYTECHNIC

ML vs. Deep Learning

- Conventional ML (대다수 머신러닝 알고리즘은 정형 데이터를 입력 데이터로 사용)
  - Feature Based Learning
    - 입력 데이터를 정형화 (Feature: 개별적이고 측정 가능한 속성 (N-Dimensional Feature))
    - Feature Engineering (Haar-Feature, HOG, etc.) <머신 러닝 알고리즘에 적합한 Feature를 도출하기 위해 많은 시간을 투자>



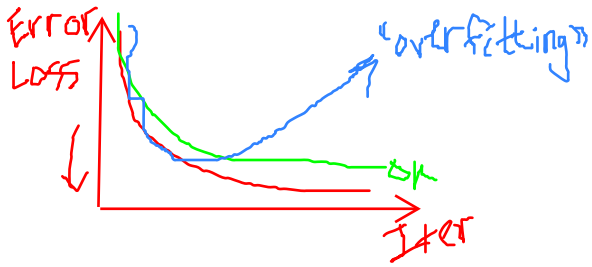


- Deep Learning
  - 전통적인 머신러닝 알고리즘은
    피처 추출을 직접 수행 후 모델을 학습
  - 딥러닝의 경우, 모델을 학습하는 과정에서 문제를 풀기 위한 적합한 피처 추출

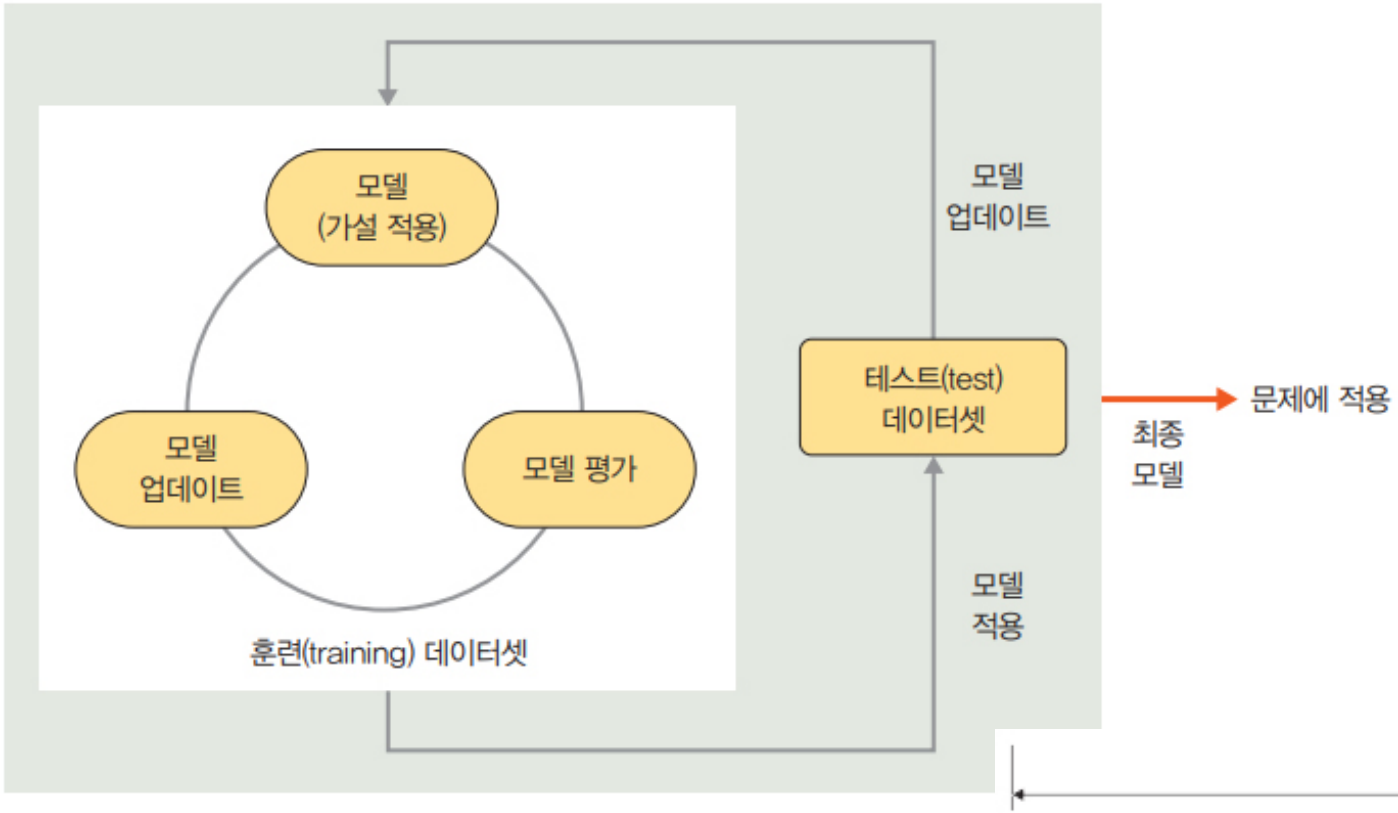


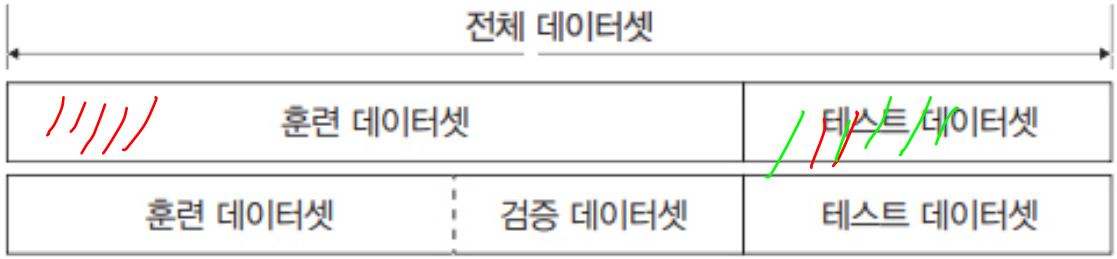
© KOPO-Dept.
AI Engineering Limited

모델 학습 *graph 고려해야함
★Error(cost)를 -minimize(최소화)가 목적



일반적인 머신러닝 모델 개발 방법 / Dataset

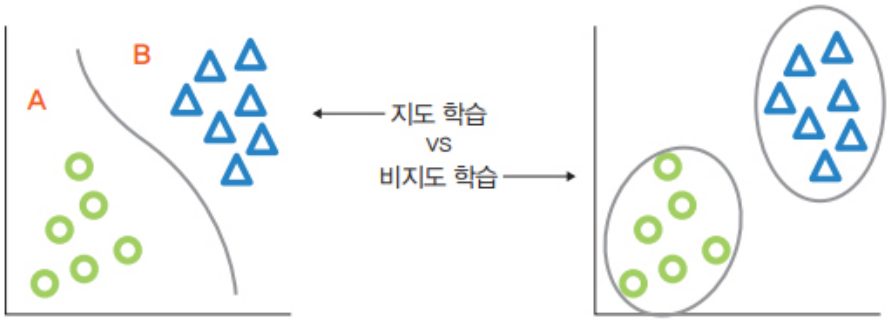


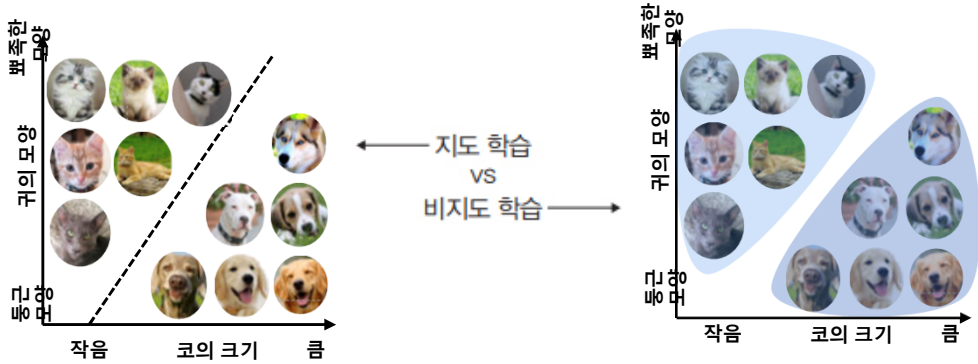


머신러닝 학습 알고리즘 종류

- 지도 학습, 비지도학습, 전이학습, 강화 학습 등으로 분류

  - 지도학습: 이름에서 알 수 있듯이 정답이 무엇인지 컴퓨터에 알려 주고 학습시키는 방법
  - 비지도학습: 정답을 알려 주지 않고 특징(다리 길이가 짧은 초식 동물)이 비슷한 데이터(토끼, 다람쥐)를 클러스터링(범주화)하여 예측하는 학습 방법





Daegu Campus of KOREA POLYTECHNIC

머신러닝 학습 알고리즘 종류

- 지도 학습, 비지도학습, 전이학습, 강화 학습 등으로 분류

  - 전이학습: 사전에 학습이 완료된 모델(pre-trained model)(사전 학습 모델)을 가지고 우리가 원하는 학습에 미세 조정 기법을 이용하여 학습시키는 방법
  - 강화학습: 자신의 행동에 대한 보상을 받으며 학습을 진행
    - 쿠키가 에이전트(agent)이며(즉, 게이머가 에이전트가 되겠죠?) 게임 배경이 환경(environment)
    - 이때 에이전트가 변화하는 환경에 따라 다른 행동(action)을 취하게 됨
    - 동전이나 젤리를 취득하는 등 행동에 따라 보상(몸집이 커짐)을 얻음
    - 강화 학습은 이러한 보상이 커지는 행동은 자주 하도록 하고, 줄어드는 행동은 덜 하도록 하여 학습을 진행



(출처: https://www.devsisters.com/ko/product/games/)

Daegu Campus of KOREA POLYTECHNIC

머신러닝 학습 알고리즘 종류

- 지도 학습, 비지도학습, 전이학습, 강화 학습 등으로 분류

구분	유형	알고리즘
지도 학습 (supervised learning)	분류(classification)	• K-최근접 이웃(K-Nearest Neighbor, KNN) • 서포트 벡터 머신(Support Vector Machine, SVM) • 결정 트리(decision tree) • 로지스틱 회귀(logistic regression)
	이미지 분류	• CNN • AlexNet • ResNet
	시계열 데이터 분석	• RNN • LSTM
	회귀(regression)	선형 회귀(linear regression)
비지도 학습 (unsupervised learning)	군집(clustering)	• K-평균 군집화(K-means clustering) • 밀도 기반 군집 분석(DBSCAN) • 가우시안 혼합 모델(Gaussian Mixture Model, GMM) • 자기 조직화 지도(Self-Organizing Map, SOM)
	차원 축소	• 오토인코더(AutoEncoder) • 주성분 분석(PCA)
전이 학습(transfer learning)	전이 학습	• 버트(BERT) • MobileNetV2
강화 학습(reinforcement learning)	–	마르코프 결정 과정(MDP)

Deep Learning 계열

Deep Learning

- Application
  - 인간 수준(이상?)의 이미지 분류기(+캡션 달기: Scene 이해)
  - 음성 인식기
  - 기계 번역
  - 자율주행 차량(이미지 인식, 신호분석, 고장 진단, 상황 판단(Decision(강화학습: 선행연구), etc.))
  - 바이오: 암 검출
  - 번역기..
  - Text 분석..
  - LLM(GPT, Llama, etc.)
  - RAG,....

Source Text

munster have signed new zealand international francis *saili* on a two-year deal . utility back *saili* , who made his all blacks debut against argentina in 2013 , will move to the province later this year after the completion of his 2015 contractual commitments . the 24-year-old currently plays for *auckland-based* super rugby side the blues and was part of the new zealand under-20 side that won the junior world championship in italy in 2011 . *saili* 's signature is something of a coup for munster and head coach anthony foley believes he will be a great addition to their backline . francis *saili* has signed a two-year deal to join munster and will link up with them later this year . ' we are really pleased that francis has committed his future to the province , ' foley told munster 's official website . ' he is a talented centre with an impressive *skill-set* and he possesses the physical attributes to excel in the northern hemisphere . ' i believe he will be a great addition to our backline and we look forward to welcoming him to munster . ' *saili* has been capped twice by new zealand and was part of the under 20 side that won the junior championship in 2011 . *saili* , who joins all black team-mates dan carter , *ma'a nonu* , conrad smith and charles *piutau* in agreeing to ply his trade in the northern hemisphere , is looking forward to a fresh challenge . he said : ' i believe this is a fantastic opportunity for me and i am fortunate to move to a club held in such high regard , with values and traditions i can relate to from my time here in the blues . ' this experience will stand to me as a player and i believe i can continue to improve and grow within the munster set-up . ' as difficult as it is to leave the blues i look forward to the exciting challenge ahead . '

Reference summary

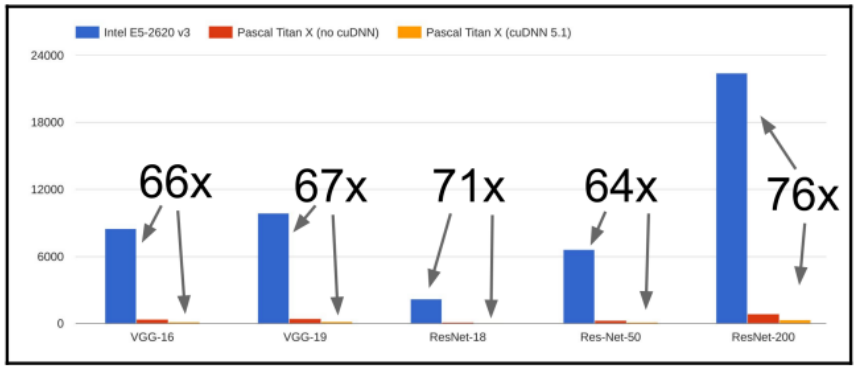
utility back francis *saili* will join up with munster later this year .
the new zealand international has signed a two-year contract .
saili made his debut for the all blacks against argentina in 2013 .



"two young girls are playing with lego toy."

Deep Learning

- 활발히 연구 되고 있으며, 쉽게 접근할 수 있는 이유....
  - Neural Network에서의 기술적 문제점 해결
  - 하드웨어 가용성



  - 데이터 및 알고리즘
    - COCO
    - ImageNet
    - CIFAR
    - Kaggle
    - KITTI
  - 딥러닝 프레임워크
    - Caffe, Tensorflow, Keras, Theano, Pytorch

© KOPO-Dept.
AI Engineering Limited

Deep Learning

- Pytorch를 포함한 여러 딥러닝 프레임워크
  - Numpy와 같은 연산을 GPU 가속 연산으로 대체
  - Deep Neural Network 구축

- Pytorch
  - 대다수 딥러닝 프레임워크는 정적인 Computational Graph 사용하고 상황에 따라 프레임워크에 종속적인 프로그래밍 스타일을 갖추고 있음.
    - 반면, 파이토치는 동적인 계산 그래프 사용(복잡한 아키텍처를 구축할 때 뛰어난 유연성 제공)
    - 클래스, 구문 및 조건, 반복문과 같은 파이썬 개념을 광범위하게 사용
    - 객체 지향 방식으로 딥러닝 알고리즘 작성 가능

- 최근…
  - ONNX (Open Neural Network Exchange)
    - e.g.) Pytorch → Caffe2, tensorflow → Pytorch

© KOPO-Dept.
AI Engineering Limited

PyTorch

- 개요
  - 파이토치(PyTorch)는 2017년 초에 공개된 딥러닝 프레임워크로 루아(Lua) 언어로 개발되었던 토치(Torch)를 페이스북에서 파이썬 버전으로 내놓은 것
  - 토치는 파이썬의 넘파이(NumPy) 라이브러리처럼 과학 연산을 위한 라이브러리로 공개되었지만 이후 발전을 거듭하면서 딥러닝 프레임워크로 발전
  - 파이썬 기반의 과학 연산 패키지로 다음 두 집단을 대상으로 함 (파이토치 공식 튜토리얼)
    - 넘파이를 대체하면서 GPU를 이용한 연산이 필요한 경우
    - 최대한의 유연성과 속도를 제공하는 딥러닝 연구 플랫폼이 필요한 경우
  - 무엇보다 주목받는 이유 중 하나는 **간결하고 빠른 구현성**에 있음

© KOPO-Dept.
AI Engineering Limited

PyTorch

- 특징: 1)**GPU**에서 2)**텐서** 조작 및 3)**동적 신경망** 구축이 가능한 프레임워크

  - **1) GPU**(Graphics Processing Unit): 연산 속도를 빠르게 하는 역할

  - **2) 텐서**(Tensor): 파이토치의 **데이터 형태**
    - 단일 데이터 형식으로 된 자료들의 다차원 행렬
    - 간단한 명령어(변수 뒤에 .cuda()를 추가)를 사용해서 GPU로 연산을 수행하게 할 수 있음

  - **3) 동적 신경망**: 훈련을 반복할 때마다 네트워크 변경이 가능한 신경망을 의미
    - 예를 들어 학습 중에 은닉층을 추가하거나 제거하는 등 모델의 네트워크 조작이 가능

```python
class DynamicNet(nn.Module):  2 usages
    def __init__(self):
        super(DynamicNet, self).__init__()
        self.fc1 = nn.Linear( in_features: 5,  out_features: 10)
        self.fc2 = nn.Linear( in_features: 10,  out_features: 1)

    def forward(self, x):
        # 매번 forward 실행 시 다른 계산 수행 가능
        if torch.rand(1).item() > 0.5:
            x = torch.relu(self.fc1(x))
        else:
            x = torch.sigmoid(self.fc1(x))
        return self.fc2(x)
```

```python
model = DynamicNet()
optimizer = optim.SGD(model.parameters(), lr=0.01)

for i in range(5):
    x = torch.randn(1, 5)
    y = torch.randn(1, 1)

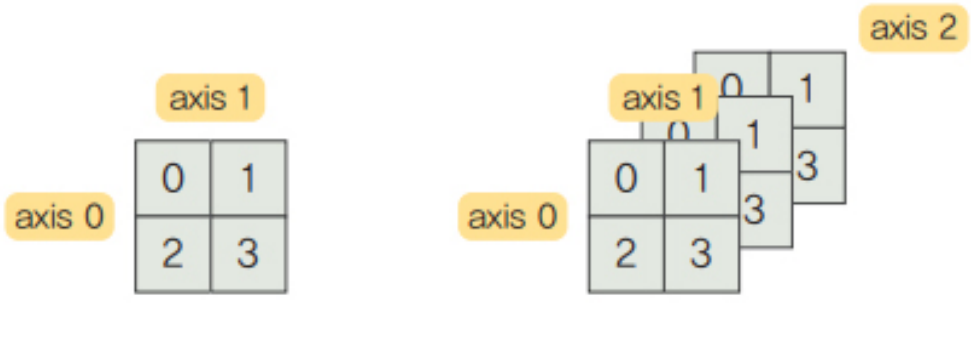
    pred = model(x)
    loss = ((pred - y) ** 2).mean()

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    print(f"Step {i + 1}, Loss: {loss.item()}")
```

PyTorch

- 특징: 벡터, 행렬, 텐서
  - 인공지능(머신 러닝/딥러닝)에서 데이터는 벡터(vector), 행렬, 텐서로 표현
  - 벡터는 [1.0, 1.1, 1.2]처럼 숫자들의 리스트로, 1차원 배열 형태
  - 행렬(matrix)은 행과 열로 표현되는 2차원 배열 형태
  - 이때 가로줄을 행(row)이라고 하며, 세로줄을 열(column)이라고 함
  - 일반적으로 텐서는 3차원 이상의 배열 형태
    - 1차원 축(행)=axis 0=벡터 (1D Tensor)
    - 2차원 축(열)=axis 1=행렬 (2D Tensor)
    - 3차원 축(채널)=axis 2=텐서 (3D Tensor)



axis 0: row

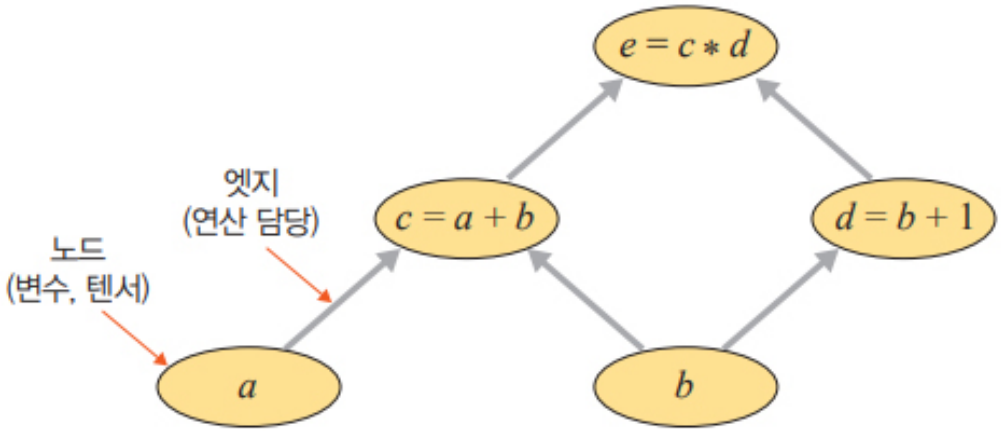
axis 1: col

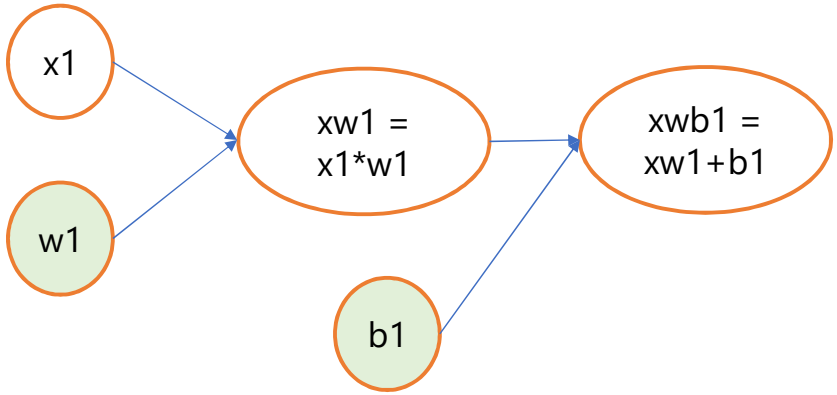
axis 2: channel

© KOPO-Dept.
AI Engineering Limited

PyTorch

- 특징: 연산 그래프
  - 연산 그래프는 방향성이 있으며 변수(예: 텐서)를 의미하는 노드와 연산(예: 곱하기, 더하기)을 담당하는 엣지로 구성
  - 노드는 변수(a, b)를 가지고 있으며 각 계산을 통해 새로운 텐서(c, d, e)를 구성할 수 있음





  - 신경망은 연산 그래프를 이용하여 계산을 수행
  - 즉, 네트워크가 학습될 때 손실 함수의 기울기가 **가중치와 바이어스를 기반으로 계산**되며, 이후 경사 하강법을 사용하여 가중치가 업데이트
  - 이때 연산 그래프를 이용하여 이 과정이 효과적으로 수행

PyTorch

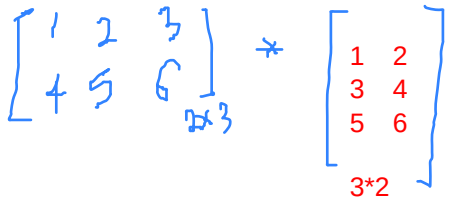
- 장점: 효율적인 계산, 낮은 CPU 활용, 직관적인 인터페이스와 낮은 진입 장벽

  - 단순함(효율적인 계산)
    - 파이썬 환경과 쉽게 통합할 수 있음
    - 디버깅이 직관적이고 간결함

  - 성능(낮은 CPU 활용)
    - 모델 훈련을 위한 CPU 사용률이 텐서플로와 비교하여 낮음
    - 학습 및 추론 속도가 빠르고 다루기 쉬움

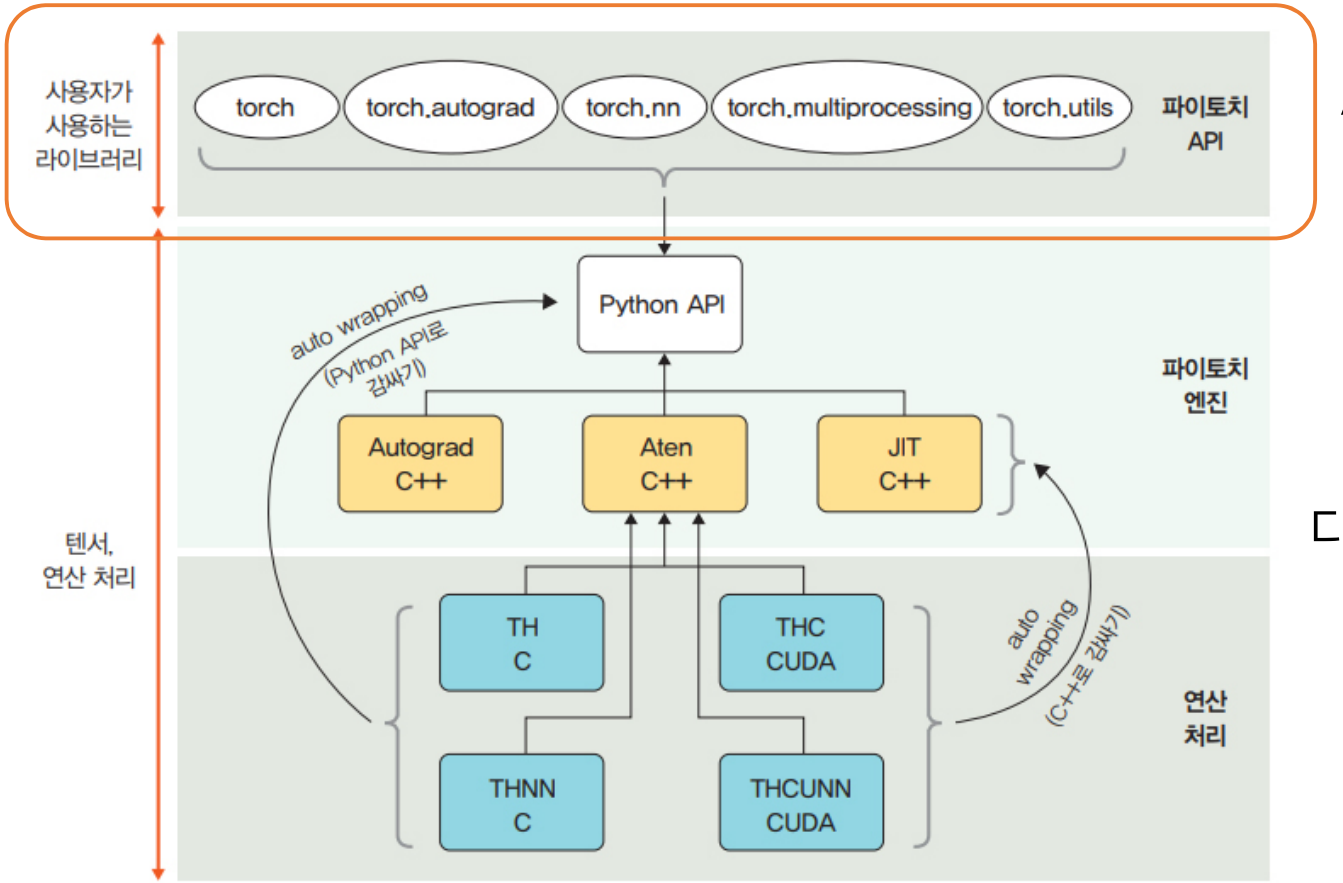
  - 직관적인 인터페이스
    - 텐서플로우처럼 잦은 API (layers → slim → estimators → tf.keras)이 없어 배우기 쉬움

© KOPO-Dept.
AI Engineering Limited



PyTorch

- 아키텍처



"우리가 사용할 거"

다차원 텐서 및 자동 미분을 처리

 © KOPO-Dept.
AI Engineering Limited

PyTorch

- Pytorch 기반 신경망 구현
  - 첫 번째 신경망 (e.g. linear regression)

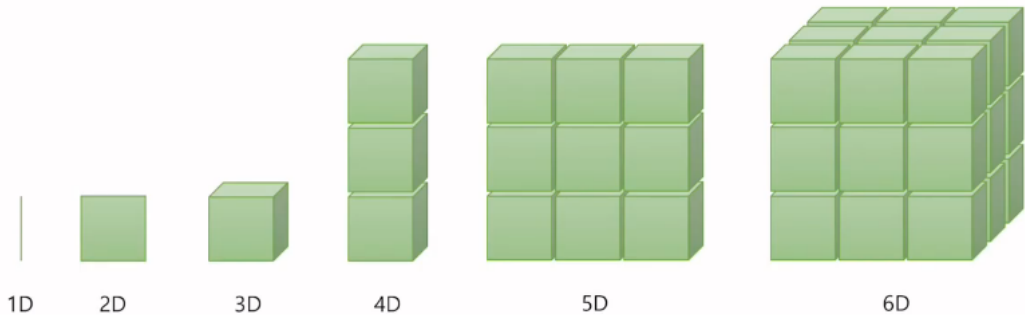
```
x,y = get_data() # x - 학습 데이터,y - 목적변수
w,b = get_weights() # w,b - 학습 파라미터
for i in range(500):
y_pred = simple_network(x) # wx + b 연산 함수
loss = loss_fn(y,y_pred) # y와 y_pred의 차이 제곱 합
if i % 50 == 0:
print(loss)
optimize(learning_rate) # 오차를 최소화하도록 w, b를 조정
```

  - get_data: 데이터 준비
    - 텐서 (Tensor), 텐서 연산
  - get_weights: 학습 파라미터 생성
    - Random Weights
  - simple_network: 네트워크 모델
    - Wx + b
  - loss_fn: 오차(Loss Fucntion)
  - optimize: 최적화
    - Autograd, Gradient, Optimizer,..

© KOPO-Dept.
AI Engineering Limited

PyTorch

- Pytorch 기반 신경망 구현
  - get_data: 데이터 준비
    - 파이토치는 데이터를 추상화하는 Tensor 제공
      - GPU에서 동작
    - Tensor: Numpy 배열과 유사한 클래스
      - 데이터를 Tensor 형태로 표현하면, 특정 연산에서 성능이 향상
      - Scalar (0-dimensional Tensor?): 그냥 real number..실수..
      - Vector (1-dimensional Tensor)
      - Matrix (2-dimensional Tensor)
      - 3-dimensional Tensor
      - 4-dimensional Tensor
      - 5-dimensional Tensor….N-dimensional Tensor
      - GPU 지원 Tensor



 © KOPO-Dept.
AI Engineering Limited

텐서를 메모리에 저장하기

- 텐서는 그것이 1차원이든 N차원이든 메모리에 저장할 때는 1차원 배열 형태가 됨
- 즉, 1차원 배열 형태여야만 메모리에 저장할 수 있음
- 변환된 1차원 배열을 스토리지(storage)라고 함
- 스토리지를 이해하기 위해서는 오프셋과 스트라이드 개념을 알아야 함

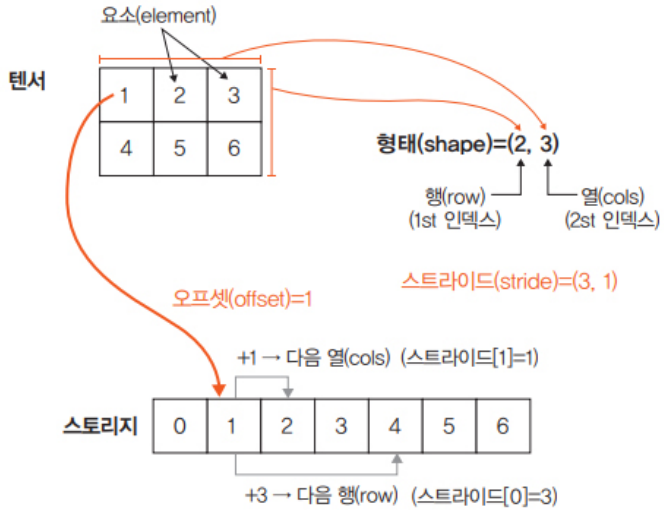
$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$ 1차원 텐서로 변환 → | 0 | 1 | 2 | 3 | 4 | 5 | 6 |

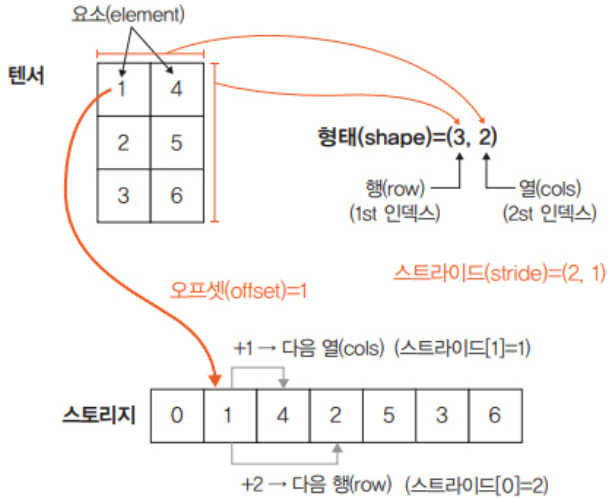
$A^T = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$ 1차원 텐서로 변환 → | 0 | 1 | 4 | 2 | 5 | 3 | 6 |

스토리지의 값들은 서로 같음

offset: 텐서에서 첫 번째 요소가 스토리지에 저장된 인덱스

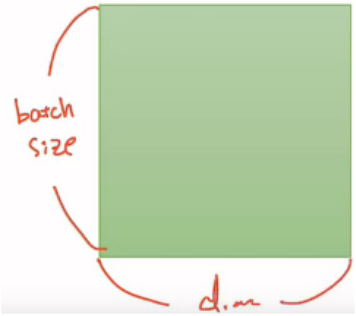
스트라이드(stride): 각 차원에 따라 다음 요소(행, 열)를 얻기 위해 건너뛰기(skip)





PyTorch

- Pytorch 기반 신경망 구현
  - get_data: 데이터 준비
    - Tensor
      - Vector (1-dimensional Tensor)
        - » 지난 주의 평균 온도 ($temp^T$ = [20, 21, 11, 15, 12, 18, 19])
        - » 학생들의 중간 고사 평균 점수 ($mid_avg_score^T$ = [80, 78, 90, … , 19])
      - Matrix(2-dimensional Tensor)
        - » Boston 집값 Dataset (8개의 Feature로 이루어진 matrix)
        - » 학생 list (N개의 Feature, 19명, 19*N Matrix)
        - » MNIST..
          - Training dataset: 60,000, Test dataset: 10,000
          - Training 시: batch size * 784 (28x28)

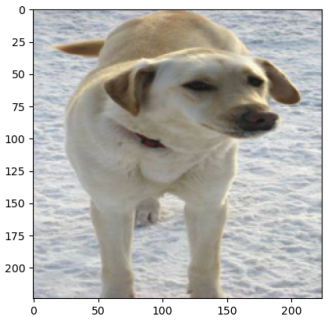


[01] CRIM	자치시(town) 별 1인당 범죄율
[02] ZN	25,000 평방피트를 초과하는 거주지역의 비율
[03] INDUS	비소매상업지역이 점유하고 있는 토지의 비율
[04] CHAS	찰스강에 대한 더미변수(강의 경계에 위치한 경우는 1, 아니면 0)
[05] NOX	10ppm 당 농축 일산화질소
[06] RM	주택 1가구당 평균 방의 개수
[07] AGE	1940년 이전에 건축된 소유주택의 비율
[08] DIS	5개의 보스턴 직업센터까지의 접근성 지수
[09] RAD	방사형 도로까지의 접근성 지수
[10] TAX	10,000 달러 당 재산세율
[11] PTRATIO	자치시(town)별 학생/교사 비율
[12] B	1000(Bk-0.63)^2, 여기서 Bk는 자치시별 흑인의 비율을 말함.
[13] LSTAT	모집단의 하위계층의 비율(%)
[14] MEDV	본인 소유의 주택가격(중앙값) (단위: $1,000)

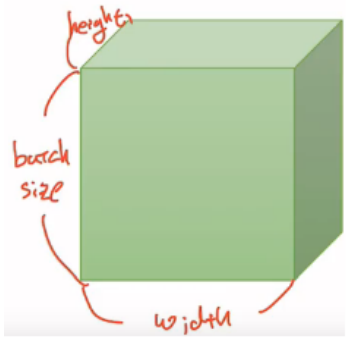
© KOPO-Dept.
AI Engineering Limited

PyTorch

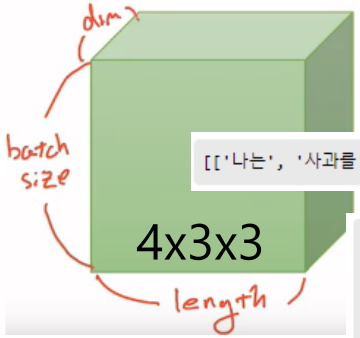
- Pytorch 기반 신경망 구현
  - get_data: 데이터 준비
    - Tensor
      - 3-dimensional Tensor
        - » 3 channel image, 1channel image + batch, NLP




```
torch.Size([224, 224, 3])
```



비전 분야에서의 3차원 텐서



3D Tensor(Typical Natural Language Processing) - NLP 분야에서의 3차원 텐서

```
[[나는 사과를 좋아해], [나는 바나나를 좋아해], [나는 사과를 싫어해], [나는 바나나를 싫어해]]
```

```
[['나는', '사과를', '좋아해'], ['나는', '바나나를', '좋아해'], ['나는', '사과를', '싫어해'], ['나는', '바나나를', '싫어해']]
```

```
'나는' = [0.1, 0.2, 0.9]
'사과를' = [0.3, 0.5, 0.1]
'바나나를' = [0.3, 0.5, 0.2]
'좋아해' = [0.7, 0.6, 0.5]
'싫어해' = [0.5, 0.6, 0.7]
```

```
[[[0.1, 0.2, 0.9], [0.3, 0.5, 0.1], [0.7, 0.6, 0.5]],
 [[0.1, 0.2, 0.9], [0.3, 0.5, 0.2], [0.7, 0.6, 0.5]],
 [[0.1, 0.2, 0.9], [0.3, 0.5, 0.1], [0.5, 0.6, 0.7]],
 [[0.1, 0.2, 0.9], [0.3, 0.5, 0.2], [0.5, 0.6, 0.7]]]
```

PyTorch

- Pytorch 기반 신경망 구현
  - get_data: 데이터 준비
    - Tensor
      - 4-dimensional Tensor
        - » 3 channel image batch
          - 64x224x224x3 (64: batch size, 224: width, 224: height, 3: channels)
        - » 한번에 한 장의 이미지를 처리하는 것보다 여러 이미지를 배치로 묶어서 처리하는 방식이 일반적임.
      - 5-dimensional Tensor
        - » 일반적인 예는 동영상 데이터
        - » e.g.) 강아지 산책 동영상 1개 (30fps: frame per second)
          - 1 x 30 x 224 x 224 x 3
          - 32 x 30 x 224 x 224 x 3 (30fps 동영상 32개)

작년부터 여기를 숙제로 내주고 있는데 위에서 잘 넘어오지 못한다.

© KOPO-Dept.
AI Engineering Limited

PyTorch

- Pytorch 기반 신경망 구현: 실습 #1
  - get_data: 데이터 준비
    - Tensor 조작 (Numpy와 유사함)
      - 1D with Pytorch
      - 2D with Pytorch
      - 3D, 4D with Pytorch
      - Broadcasting
      - 자주 사용되는 기능..
        - » Matrix Multiplication, Multiplication(element-wise 곱), mean, sum, etc.

© KOPO-Dept.
AI Engineering Limited

Deep Learning

- Pytorch 기반 신경망 구현: 실습 #1
  - Tensor 이해 등

```
import time
from torch import cuda


print(torch.__version__)

# vector: 1-dimensional Tensor
# 텐서를 만들기 위해 torch.FloatTensor() <-dimension 입력(like numpy)
# 지난 중의 평균 온도를 벡터에 저장
temp = torch.FloatTensor(
    [15, 17, 19.2, 22.3, 20, 19, 16]
)
print(temp.size(), temp.dim())
# indexing, slicing
print(f'월, 화 평균온도는: {temp[0]}, {temp[1]}')
print(f'화~목 평균온도는: {temp[1:4]}')

# Matrix: 2-dimensional Tensor
# 파이토치로 2차원 행렬(2-dimensional tensor)
```


© KOPO-Dept.
AI Engineering Limited

Thank you

Q&A

www.kopo.ac.kr
jsshin7@kopo.ac.kr

 © KOPO-Dept.
AI Engineering Limited